Svivva Play – Step■by■Step Strategic Prompts (Replit■Build Ready)

Version 2 • 2026-02-12

Goal: Add 5 “Modes” to Svivva Play inside the instrument UI, all powered by Svivva API Creator.
Every mode must output professional■quality results (Udio/Suno■level). If a pipeline cannot meet the Quality Gates below, the mode must be disabled or flagged “Beta/Unavailable” (do not ship weaker approximations).

Global Non■Negotiables

All modes share these requirements:
• Input: user■imported audio (and optional prompt / reference link / “in■my■head” vocal sketch).
• Analysis: key, tempo, chords, structure (sections), downbeats, instrument hints.
• Output: (1) separated multitrack audio stems, (2) per■stem MIDI, (3) per■stem patch/sound metadata, (4) session JSON to reload in Svivva Play UI.
• Scale intelligence: scale lookup + scale constraints + reharmonization aware.
• “Indian Meend” toggle: continuous pitch bend/glide expression on melodic parts (MPE■friendly).
• UX: one “Generate” button + advanced drawer; progress, cancel, resume, version history.
• Privacy: do not store user audio unless user opts■in; store minimal embeddings if needed.

Quality Gates (Ship/No■Ship)

A mode is considered “Professional” only if it passes:
1) Realism: instrument render is indistinguishable from high■quality sample libraries in blind A/B with 5 listeners.
2) Separation: stems are phase■coherent, minimal bleed, re■mixable without artifacts.
3) Musicality: phrases have coherent development; rhythm aligns to detected grid; harmonies respect key/chords unless user selects reharmonize.
4) Consistency: same seed + same settings yields reproducible results; export is deterministic per seed.
5) Latency: “Preview” result ≤ 30–60s; “Full render” can be longer but must show staged progress and allow cancel.

Shared Build Blueprint (Use for Every Mode)

A. UI Flow – Instrument Panel

Step A1) User selects Mode (Composition / Interpolation / Chord Player / Solo Prompt / Analog Patch / Ensemble).
Step A2) User imports audio (or records “In■My■Head”).
Step A3) UI calls /analyze and displays: Key, BPM, time signature, chord timeline, section map.
Step A4) User selects Style (or writes prompt).
Step A5) User chooses “Preview” or “Full Render.”
Step A6) UI shows streaming previews + stem list. User can mute/solo stems, drag levels, re■generate individual stems.
Step A7) Export: ZIP (stems + MIDI + session.json + patch.json).

B. Backend Service Stages (Pipeline)

Every “Generate” is a pipeline with checkpoints:
Stage 0: Validate inputs (format, duration, loudness, copyright flags if needed).
Stage 1: Analysis → key/BPM/chords/structure + embeddings.
Stage 2: Plan → arrangement plan, instrument roster, register allocation, motif rules, harmony plan.
Stage 3: Generate MIDI → per■stem MIDI with expression (CC, velocity, MPE).
Stage 4: Render Audio → either (a) diffusion/transformer audio model or (b) high■quality sampler engine (SFZ/Kontakt■like) + performance rendering.
Stage 5: Stem Separation/Post → ensure stems are clean, aligned, normalized.
Stage 6: Package → create exports + session JSON.
Stage 7: QA → automated checks (clipping, alignment, key match), then mark READY.

C. Universal Data Contracts (JSON)

```
// session.json (minimum)
{
"project_id": "...",
"source_audio": {"sr": 48000, "duration_s": 123.4},
"analysis": {
"bpm": 124.0,
"time_signature": "4/4",
"key": "E minor",
"chords": [{"t0":0.0,"t1":2.0,"symbol":"Em9"}, ...],
"sections": [{"name":"A","t0":0,"t1":32}, {"name":"B","t0":32,"t1":64}],
"downbeats": [0.0, 1.935, ...]
},
"mode": "composition",
"style": {"preset":"interlocking_minimalism","prompt":"...","seed":12345},
"stems": [
{"name":"Marimba
1","audio":"stems/marimba1.wav","midi":"midi/marimba1.mid","pan":-25,"gain_db":-3.0,
"expression":{"mpe":true,"meend":false}}
],
"exports": {"zip":"exports/project.zip"}
}

// patch.json (synth mode)
{
"target": {"synth_family":"subtractive_analog"},
"oscillators":[{"shape":"saw","oct":0,"fine":0.0,"mix":0.65}, ...],
```

```
"filter":{"type":"ladder_lp","cutoff_hz":1200,"resonance":0.35,"drive":0.2},
"env":{"amp":{"A":0.01,"D":0.35,"S":0.7,"R":0.25},
"filter":{"A":0.02,"D":0.45,"S":0.2,"R":0.3,"amount":0.55}},
"lfo":[{"wave":"sine","rate_hz":4.2,"dest":"pitch","amount":0.02}],
"fx":[{"type":"chorus","mix":0.25},{"type":"delay","time":"1/8","mix":0.18}],
"mappings":{"vst3":{"param_ids":{...}},"sysex":{"enabled":false}}
}
```

Prompt System (Replit■Friendly Templates)

Use these templates as “LLM Orchestrator” prompts.
Implementation: Svivva API Creator should expose a “Prompt Runner” that fills placeholders and logs outputs to pipeline stages.

Template 1: Analysis Orchestrator Prompt

```
SYSTEM:
You are SvivvaPlayAnalysis. Output ONLY valid JSON that conforms to
analysis.schema.json.

DEVELOPER:
Given user imported audio features (bpm candidates, chroma, onset map, embeddings)
produce:
- final bpm, time signature, key (with confidence)
- chord timeline (t0,t1,symbol,roman_numeral,confidence)
- section boundaries (A,B,C...) with bar ranges
- “style_compatibility” hints (what modes/styles fit)
- performance grid (downbeats, bars)

USER INPUT (JSON):
{ "audio_meta": {{AUDIO_META}}, "features": {{FEATURES}}, "user_hint": "{{USER_HINT}}" }

OUTPUT (JSON ONLY):
{ "bpm":..., "time_signature":..., "key":..., "chords":[...], "sections":[...],
"downbeats":[...], "style_compatibility":[...] }
```

Template 2: Arrangement Planner Prompt

```
SYSTEM:
You are SvivvaPlayPlanner. Output ONLY JSON plan. Do not generate MIDI yet.

DEVELOPER:
Inputs: analysis JSON + selected mode + style preset + constraints.
Return: instrument roster, stem roles, motif rules, harmony rules, density curve,
panning plan, dynamics plan.
Must include: "meend_applicable_stems" list.

USER INPUT (JSON):
{
"analysis": {{ANALYSIS_JSON}},
"mode": "{{MODE}}",
"style_preset": "{{STYLE_PRESET}}",
"user_prompt": "{{USER_PROMPT}}",
"constraints": {{CONSTRAINTS_JSON}}
}

OUTPUT (JSON ONLY):
{
"stems":[{"name":"...","role":"bass|harmony|melody|perc|pad|vocal","register":"low|mid|h
igh",
"instrument_hint":"...", "density_curve":[...], "pan":-30}],
"motif_rules":[...],
"harmony_rules":[...],
"dynamics":[...],
"meend_applicable_stems":["..."]
```

```
}
```

Template 3: MIDI Generator Prompt

```
SYSTEM:
You are SvivvaPlayMIDIGen. Output ONLY JSON MIDI events (note, start, dur, vel, channel,
cc, pitchbend).

DEVELOPER:
Generate per-stem MIDI aligned to analysis.downbeats + chord timeline.
Respect: scale constraints, motif rules, density curve, articulation plan, meend toggle.
Provide BOTH:
- "preview" (8–16 bars)
- "full" (entire duration or selected range)

USER INPUT (JSON):
{ "analysis": {{ANALYSIS_JSON}}, "plan": {{PLAN_JSON}}, "range": {{RANGE}}, "settings":
{{SETTINGS}} }

OUTPUT (JSON ONLY):
{
"stems":[{"name":"...","midi_events":[...],"expression":{"cc":[...],"pitchbend":[...]}}]
}
```

Template 4: Audio Render Prompt (Model Router)

```
SYSTEM:
You are SvivvaPlayRenderRouter. Output ONLY JSON routing instructions.

DEVELOPER:
Choose rendering method per stem:
A) Generative audio model (diffusion/transformer) OR
B) Performance sampler engine using curated pro sample packs.
Decision must optimize realism and speed and meet Quality Gates.

USER INPUT (JSON):
{ "analysis": {{ANALYSIS_JSON}}, "plan": {{PLAN_JSON}}, "midi": {{MIDI_JSON}}, "assets":
{{ASSET_CATALOG}} }

OUTPUT (JSON ONLY):
{
"routes":[
{"stem":"Marimba
1","method":"sampler","preset":"ProMarimbaLegato","articulation_map":{...}},
{"stem":"Vocal","method":"gen_audio","model":"svivva-vocalgen-vX","conditioning":{...}}
]
}
```

Note: Replit implementation can treat the above prompts as string constants and fill placeholders from pipeline stage outputs. The actual audio generation should be done by dedicated models/services, not the LLM.

Mode 1 — Composition

Purpose: Generate a new composition that *matches* the imported audio's harmonic language and feel, while applying advanced compositional processes (minimal interlocking counterpoint, hocketing textures, tone■row etudes, sonification, automata/chaos, Tom Johnson■like formal systems).
Output: multitrack stems + MIDI + session.

UI Steps

1) User imports audio and chooses "Composition."
2) UI runs /analyze → shows chord timeline + section map.
3) User selects a "Process Preset" (Interlocking Minimalism / Hocketed Texture / Phase Motion / Tone Row / Sonification / Automata / Rational Melody / Counting / Combinatorial).
4) User sets: Density, Complexity, Section Length, Instrument Palette, Meend toggle, "Match Chords Strictly vs Reharmonize."
5) Preview generates 8–16 bars (per section). User can regenerate just one stem or one section.
6) Full render generates entire track with consistent motif development and panning.

API Endpoints (Suggested)

```
/POST /svivva_play/analyze
body: {audio_url|upload_id, user_hint}
returns: analysis.json

/POST /svivva_play/compose/plan
body: {analysis, style_preset, user_prompt, constraints}
returns: plan.json

/POST /svivva_play/compose/midi
body: {analysis, plan, range, settings}
returns: midi.json

/POST /svivva_play/render
body: {analysis, plan, midi, asset_catalog, render_quality: preview|full}
returns: stems + session.json

/POST /svivva_play/export
body: {project_id}
returns: export zip url
```

Settings (Keep concise in UI)

• Density (0–100)
• Complexity (0–100)
• Harmony: Match / Reharmonize
• Scale: Auto / Select scale
• Meend: Off/On (melody stems only)
• Panning: Auto / Manual spread
• Seed

Style Presets

• Generated MIDI aligns to detected BPM/grid within ±10ms.
• Chords respected when Harmony=Match: non■chord tones allowed only as passing/neighbor with resolution rules.
• Stems export clean, aligned, and remixable (no stereo drift, no clipping).
• Preview and full output share same motif DNA (not random mismatch).
• Meend toggle produces continuous bends (no stair■stepping).

Acceptance Criteria

• Interlocking Minimal Counterpoint (two+ melodic lines that interlock; avoid collisions; emphasize complementary rhythms)
• Hocketed Texture (short notes distributed across instruments; staggered entrances; strict alternation %)
• Phase Motion (one part gradually shifts timing; maintain downbeat anchors)
• Tone■Row Etude (12■tone row with transformations; optional tonal anchors)
• Sonification (map feature → pitch/rhythm/dynamics; user chooses mapping)
• Automata/Chaos (rule■based state machine; reproducible with seed)
• Rational Melody (limited pitch set; explicit counting rules)
• Counting Process (notes generated by counts; audible structure)
• Combinatorial Harmony (catalog chord set; combinatorial traversal)

Special Notes

Do NOT name artist presets in UI; use generalized labels.
For “Tom■Johnson■like” processes: expose “Rules” in an Advanced drawer so users can edit counts/automata tables.

Mode■Specific Prompt Add■Ons

```
// Planner add-on (append to Template 2 as constraints)
"process_constraints": {
"interlocking": {"avoid_unison": true, "min_offset_ms": 60, "max_density": 0.75},
"hocketing": {"alternation_percent": 70, "max_note_len_beats": 0.5},
"phase": {"shift_per_8_bars_ms": 12, "anchor_every_bars": 4},
"tone_row": {"row":"AUTO|USER", "allowed_transforms":["P","I","R","RI"], "tonal_anchor":
false}
}

// MIDI rules add-on (append to Template 3)
"voice_leading": {"max_leap_semitones": 7, "prefer_stepwise": true},
"ornamentation": {"passing_tones": true, "neighbor_tones": true, "approach_tones":
"diatonic"},
"panning_plan": {{PANNING_PLAN}}
```

Mode 2 — Interpolation / Sample Match (Style Transfer Loops)

Purpose: Recreate the imported audio as tight loops and stems, then “translate” it into a different style while keeping recognizable rhythmic/harmonic DNA.
Output: loop■perfect stems + MIDI + editable multitrack regeneration prompts.

UI Steps

1) User imports audio, selects a region (loop start/end snapped to bars).
2) /analyze extracts tempo + chord grid; UI displays loop length in bars.
3) User selects Target Style (genre preset or free prompt).
4) Preview returns 2–4 loop variations; user picks one, then “Expand” to full arrangement.
5) UI exposes “Style Strength” slider and “Keep Original Harmony” toggle.

API Endpoints (Suggested)

```
/POST /svivva_play/interpolate/plan
/POST /svivva_play/interpolate/midi
/POST /svivva_play/interpolate/render
```

Settings (Keep concise in UI)

• Loop Length (bars)
• Target Style (preset/prompt)
• Style Strength (0–100)
• Keep Harmony (on/off)
• Groove Quantize (light/med/heavy)
• Seed

Style Presets

• Loop must be click■free at boundaries (zero■crossing or crossfade).
• Stems remain phase■aligned across loop repeats.
• Style Strength behaves predictably (0 = near original; 100 = strong transformation).

Acceptance Criteria

• Genre Transfer (e.g., soul → synth■funk)
• Texture Transfer (same harmony, new timbres)
• Rhythm Transfer (new groove, same motifs)

Special Notes

This is not “cheap” EQ/FX. Must be true re■generation using high■quality instrument models/samplers.

Mode■Specific Prompt Add■Ons

```
// Analysis add-on: loop grid
"loop": {"t0": {{LOOP_T0}}, "t1": {{LOOP_T1}}, "bars": {{BARS}}}

// Planner add-on: style transfer
```

"transfer": {"keep_harmony": {{KEEP_HARMONY}}, "style_strength": {{STYLE_STRENGTH}},
"target_prompt":"{{TARGET_PROMPT}}"}

Mode 3 — Chord Player (Advanced Reharm + Performance)

Purpose: Generate an advanced chord layer over imported audio, with professional voicings and comping patterns.
Output: chord■stem audio + MIDI (plus optional bass re■write).

UI Steps

1) Import audio → /analyze produces chord timeline with confidence.
2) User chooses “Chord Engine” preset (Neo■Soul Extended / Jazz■Fusion / Sophisticated Pop / Cinematic Modal / Complex Jazz).
3) User selects “Comping Pattern”: sustained pads / rhythmic stabs / arpeggiated / guitar■like.
4) Generate Preview (8 bars), then Full.
5) User can lock “Top Note Melody” or “Bass Note” for consistent voice leading.

API Endpoints (Suggested)

```
/POST /svivva_play/chords/plan
/POST /svivva_play/chords/midi
/POST /svivva_play/chords/render
```

Settings (Keep concise in UI)

• Chord Engine preset
• Comping Pattern
• Tension (0–100)
• Root Movement (stable ↔ adventurous)
• Voice Leading (smooth ↔ jumpy)
• Scale Override
• Meend (usually off for chords)

Style Presets

• Chords must fit the timeline; when “Reharmonize” enabled, must still cadence logically into section boundaries.
• Voicings must be playable (range limits, no impossible clusters unless user requests).
• MIDI must include velocities reflecting comping groove.

Acceptance Criteria

• Neo■Soul Extended Harmony (9/11/13, slash chords, smooth voice leading)
• Jazz■Fusion Voicing Engine (quartal stacks, upper structures)
• Sophisticated Pop Harmony (color chords without losing hook)
• Cinematic Modal Harmony (pedal points, modal interchange)
• Complex Jazz Matrix (tritone subs, backdoor dominants, chromatic approach chords)

Special Notes

UI should show chord symbols and allow per■bar edits; regeneration respects user edits (lock).

Mode■Specific Prompt Add■Ons

```
// Planner add-on: reharm controls
"reharm": {
"tension": {{TENSION}},
"root_movement": {{ROOT_MOVEMENT}},
"voice_leading": "{{VOICE_LEADING}}",
"locked_notes": {{LOCKED_NOTES}}
}

// MIDI add-on: comping
"comping": {"pattern":"{{PATTERN}}","swing":{{SWING}},"humanize_ms":{{HUMANIZE}}}
```

Mode 4 — Solo Prompt (AI Soloist + Vocal Scat)

Purpose: Generate an intelligent solo over imported audio (instrumental or vocal scat), with phrasing, development, and harmonic awareness.
Output: solo stem audio + MIDI + harmony■solo option.

UI Steps

1) Import audio → /analyze chord grid + section map.
2) User chooses Solo Type (Lead Instrument / Vocal Scat / Harmony Solo).
3) User chooses Style preset and sets “Risk” (inside ↔ outside).
4) Preview returns 2–3 takes; user selects and “Develop” (adds motif callbacks across sections).
5) Full render outputs consistent arc (intro → peak → resolve).

API Endpoints (Suggested)

```
/POST /svivva_play/solo/plan
/POST /svivva_play/solo/midi
/POST /svivva_play/solo/render
```

Settings (Keep concise in UI)

• Solo Type: Instrument | Vocal Scat | Harmony Solo
• Style preset
• Risk (0–100)
• Density (0–100)
• Meend toggle (on for raga■inspired)
• Call■and■Response (on/off)
• Seed

Style Presets

• Phrasing must breathe (rests) and develop motifs (no random noodling).
• Must respect chords when Risk low; when Risk high, must still resolve at cadences.
• Vocal scat must avoid uncanny artifacts; if not pro■level, disable scat option.

Acceptance Criteria

• Modal Jazz Lead (smooth motifs, space, dynamics)
• Bebop Improviser (fast chromatic enclosures, strong resolutions)
• Blues Fusion Lead (bends, vibrato, pentatonic + color tones)
• Raga■Inspired (ornaments, meend, drone sensitivity)
• Vocal Scat (syllable engine + harmony stacks)

Special Notes

For vocal scat, build a syllable■phoneme plan stage before rendering audio (to keep intelligible “doo/dah” style).

Mode■Specific Prompt Add■Ons

```
// Planner add-on: solo arc
"solo_arc": {"sections":[{"name":"A","energy":0.35},{"name":"B","energy":0.75},{"name":"
C","energy":0.55}]},
```

```
"risk": {{RISK}},
"call_response": {{CALL_RESPONSE}}

// Vocal add-on (if scat)
"scat": {"syllable_set":["doo","dah","ba","ya","na"],"clarity":0.8,"stack_harmonies":{{H
ARMONY_SOLO}}}
```

Mode 5 — Analog Synth Patch Creator (Universal Patch + Files)

Purpose: From imported audio (or song prompt), generate a patch that recreates the timbre on analog synths and VSTs.
Output: patch.json + human■readable instructions + optional SysEx/VST parameter mappings.

UI Steps

1) User imports a short reference clip (1–10s) or selects region.
2) /analyze extracts timbre descriptors (spectral centroid, harmonicity, envelope, modulation).
3) User selects Synth Family: Subtractive / FM / Wavetable / Granular.
4) Generate patch: UI shows macro knobs (Brightness, Movement, Bite, Space).
5) Export includes: patch.json + per■synth mapping templates (where available).

API Endpoints (Suggested)

```
/POST /svivva_play/patch/analyze_timbre
/POST /svivva_play/patch/design
/POST /svivva_play/patch/export
```

Settings (Keep concise in UI)

• Synth Family
• Brightness / Movement / Bite / Space macros
• Unison (voices/detune)
• Mono/Poly
• Mod Depth
• Seed

Style Presets

• Patch must recreate envelope + spectral profile within tolerance (define metrics: MFCC distance, spectral slope).
• Instructions must be actionable knob■by■knob.
• If universal mapping isn’t possible, output must clearly state supported synth targets.

Acceptance Criteria

• Subtractive Analog (ladder/SEM filters)
• FM Electric / FM Metallic
• Wavetable Modern
• Granular Texture Pad

Special Notes

“Universal for all synths” requires a standard abstraction layer. Implement “SvivvaPatchSpec” and then write adapters per synth (VST3 param IDs, MIDI CC, SysEx when possible).

Mode■Specific Prompt Add■Ons

```
// Patch prompt (use as a separate LLM step)
SYSTEM: You are SvivvaPatchDesigner. Output ONLY patch.json matching patch.schema.json.
```

DEVELOPER: Create a synth-agnostic patch spec (SvivvaPatchSpec) plus optional adapters.
USER: {"timbre_features": {{TIMBRE}}, "family":"{{FAMILY}}", "macros": {{MACROS}}}

Bonus — Ensemble Concerto Builder (Full Band/Orchestra + Vocalist)

Purpose: Build a full ensemble piece (up to 40■piece orchestra or genre■accurate band) based on imported audio.
This is compute■heavy and should support Preview→Full staging.
Output: many stems, articulation■aware MIDI, realistic vocalist generation.

Step■by■Step

1) Import audio → /analyze.
2) Choose Ensemble preset (Cinematic Orchestra / 60s Soul / 80s Synth■Funk / 70s Sophisticated Jazz■Rock / 90s R&B; / Modern Hybrid).
3) Choose “Vocalist” toggle + lyric mode: (a) instrumental vocalizations, (b) user■provided lyrics, (c) placeholder phonemes.
4) Planner generates: instrumentation, voicings, articulation map, dynamics curve, panning stage plot.
5) MIDI generation produces per■section articulations (legato/staccato/tremolo, etc).
6) Render stage uses pro sample libraries and a dedicated vocal model; create stems.
7) QA: articulation realism, reverb coherence, mix balance.
8) Export ZIP + session.json.

Ensemble Planner Prompt Add■On

```
// Append to Template 2:
"ensemble": {
"size": 40,
"sections": ["strings","woodwinds","brass","perc","keys","bass","guitars","vocal"],
"articulations_required": true,
"dynamics": {"pp_to_ff": true, "crescendos": true},
"stage_plot": {"strings_front":true,"brass_back":true,"perc_right":true},
"vocalist": {"enabled": {{VOCAL_ON}}, "mode":"phoneme|lyrics|vocalize",
"style":"{{VOCAL_STYLE}}"}
}
```

Acceptance Criteria

• Orchestra/band sounds realistic (sample■library quality), with convincing articulations and dynamics.
• Vocalist must be natural; if not, disable vocalist for that preset.
• Stems are mix■ready: coherent ambience, no harsh artifacts, consistent loudness targets.

Cross■Mode Feature — Vocal Swap (Audimee■Like)

Purpose: Replace a vocal stem with a new voice while preserving timing, expression, and phrasing.
Integrate as a post■process available in all modes whenever a vocal stem exists.

Pipeline Steps

1) Isolate vocal stem (or use generated vocal).
2) Extract: pitch contour, timing, phoneme alignment, dynamics, formants.
3) Apply voice conversion model (timbre transfer) with formant preservation + breath/noise modeling.
4) Re■mix: match loudness, reverb send, and stereo placement.
5) Export: swapped vocal stem + original kept as alternate take.

API Endpoints

```
/POST /svivva_play/vocal_swap/analyze
/POST /svivva_play/vocal_swap/convert
/POST /svivva_play/vocal_swap/render
```

Prompt Add■On (Conversion Director)

```
SYSTEM: You are SvivvaVocalSwapDirector. Output ONLY JSON.
DEVELOPER: Choose conversion settings to preserve expression and avoid artifacts.
USER: {"source_vocal_features": {{VOCAL_FEATS}}, "target_voice_id":"{{VOICE_ID}}",
"goal":"{{GOAL}}"}
OUTPUT:
{"formant_lock":0.8,"breath_preserve":true,"sibilance_control":0.6,"reverb_match":true}
```

Guided Prompt Builder (So Users Can Request ANY Style)

Implement a “Guided Build” wizard that generates a structured prompt from short user choices.
This keeps prompts concise but highly specific—ideal for Replit + Svivva API Creator.

Wizard Steps → Structured Prompt JSON

Step 1: Choose Outcome (new composition / reharmonize / solo / style transfer / ensemble / patch).
Step 2: Choose Reference (imported audio region + optional reference track link).
Step 3: Choose Style (preset + optional text).
Step 4: Choose Instruments (palette).
Step 5: Choose Harmony policy (match vs reharm).
Step 6: Choose Scale (auto or user selection) + Meend.
Step 7: Choose Length and Form (AABA, through■composed, etc).
Step 8: Generate prompt JSON and lock it into the session.

```
// guided_prompt.json
{
"outcome":"composition",
"reference":{"region":{"t0":0,"t1":32},"match_energy":true},
"style":{"preset":"hocketed_texture","notes":"airy, bright, shimmering"},
"instruments":["marimba","piano","strings_small","perc_light"],
"harmony":{"policy":"match","tension":35},
"scale":{"mode":"auto","override":null},
"expression":{"meend":true,"mpe":true},
"form":{"bars":128,"sections":["A","A","B","A"],"build_curve":"slow_peak"},
"render":{"preview_bars":16,"quality":"pro"},
"seed":12345
}
```

Replit Build Checklist

- Store all prompts as versioned templates in the repo (templates/*.prompt).
- Implement pipeline workers (queue) for stages; return status via SSE/WebSocket.
- Cache analysis per upload_id; regenerate only when input changes.
- Allow per■stem regeneration using the same plan + locked stems.
- Export ZIP with deterministic filenames.
- Add automated QA checks before marking READY.